

Design and Implementation of a 4-Requester Round Robin Arbiter Using Verilog HDL

Abstract

A Round Robin Arbiter is a hardware component used to allocate a shared resource, such as a bus or memory interface, among multiple requesters in a fair and efficient manner. Unlike fixed-priority arbiters, which may continuously favor higher-priority requesters, a round robin arbiter rotates the priority after each completed transaction, ensuring that every requester eventually receives access.

This project presents the design and implementation of a synchronous 4-requester Round Robin Arbiter using Verilog HDL. The arbiter employs a Finite State Machine (FSM) to control the arbitration process and maintains fairness using a rotating priority mechanism implemented with a mask register. The design was verified through simulation using Xilinx Vivado.

1. Introduction

Modern digital systems frequently contain multiple devices that need to access a common hardware resource such as a shared communication bus, memory controller, or peripheral interface. Since only one device can use the resource at a time, an arbitration mechanism is required to determine which requester receives access.

A Round Robin Arbiter provides a simple and fair scheduling technique by granting access to requesters in a cyclic order. After servicing one requester, its priority is moved to the lowest level, allowing the next requester in sequence to receive service during the following arbitration cycle. This prevents starvation and ensures that all requesters receive fair access to the shared resource.

The objective of this project is to implement a hardware-based Round Robin Arbiter capable of handling four requesters using Verilog HDL.

2. Objectives

The objectives of this project are:

- Design a synchronous 4-requester Round Robin Arbiter.
 - Implement arbitration using Verilog HDL.
 - Use a Finite State Machine (FSM) to control arbitration.
 - Ensure that only one requester receives a grant at any time.
 - Implement rotating priority to prevent starvation.
 - Verify the design through behavioral simulation in Xilinx Vivado.
-

3. System Overview

The arbiter receives request signals from four devices. Whenever one or more devices request access to the shared resource, the arbiter selects one requester based on the current round robin priority.

The granted requester retains ownership of the shared resource until it completes its transaction and deasserts its request signal. After completion, the arbiter updates its priority pointer and grants the resource to the next eligible requester.

The design consists of three major functional blocks:

- Finite State Machine (FSM)
- Arbitration Logic

- Grant Register

Together, these blocks perform request evaluation, grant generation, and priority rotation.

4. Input and Output Signals

Inputs

Signal	Description
CLK	System clock
RST_I	Active-high synchronous reset
CYC0	Request signal from Device 0
CYC1	Request signal from Device 1
CYC2	Request signal from Device 2
CYC3	Request signal from Device 3

Outputs

Signal	Description
GNT0	Grant for Device 0
GNT1	Grant for Device 1
GNT2	Grant for Device 2
GNT3	Grant for Device 3
COMCYC	Indicates that the shared resource is occupied
GNT[1:0]	Encoded grant output

5. Finite State Machine Design

The arbiter is controlled using a three-state Finite State Machine.

IDLE

The arbiter remains in the IDLE state while no requests are active. During this state, all grant outputs remain low.

If one or more requests become active, the FSM transitions to the ARBITRATE state.

ARBITRATE

In this state, the arbitration logic examines all active requests together with the current round robin priority stored in the mask register.

The highest-priority active requester is selected and its corresponding grant signal is asserted.

After a grant is issued, the FSM transitions to the HOLD state.

HOLD

During the HOLD state, the granted requester retains ownership of the shared resource.

The arbiter continuously monitors the request signal of the granted device.

Once the requester releases the bus by deasserting its request signal, the grant is removed, the round robin pointer is updated, and the FSM returns to the IDLE state to begin the next arbitration cycle.

6. Arbitration Logic

The arbitration logic is responsible for selecting the next requester based on the current round robin priority.

Unlike fixed-priority arbitration, the starting point of priority changes after every completed transaction. For example, if Device 2 was the most recently serviced requester, the next arbitration cycle begins by checking Device 3 first, followed by Device 0, Device 1, and finally Device 2.

This rotating priority ensures that every requester eventually receives access to the shared resource, preventing starvation.

7. Grant Register

The grant register stores the currently selected requester.

Once a requester receives a grant, the corresponding grant signal remains asserted until that requester releases the bus.

This prevents the arbiter from changing ownership while a transaction is still in progress.

The grant register therefore guarantees stable bus ownership throughout the entire transaction.

8. Round Robin Priority Update

The arbiter maintains a mask register that stores the encoded value of the previously granted requester.

After each completed transaction, the mask register is updated with the current grant.

During the next arbitration cycle, the search for the next requester begins immediately after the previously serviced requester.

This rotating priority mechanism provides equal opportunity for all requesters to access the shared resource.

9. Simulation and Verification

Behavioral simulation was performed using Xilinx Vivado.

The following scenarios were verified:

- Reset operation
- Single requester
- Multiple simultaneous requests
- Grant hold during active transactions
- Round robin priority rotation
- All requesters requesting simultaneously

Simulation waveforms confirmed that:

- Only one grant was active at any given time.
 - Grants remained active until the requester released the resource.
 - The arbiter correctly rotated the priority after every completed transaction.
 - No requester experienced starvation.
-

10. Results

Simulation results demonstrate that the implemented arbiter successfully performs fair resource allocation among four competing requesters.

The FSM correctly manages the arbitration process while the rotating priority mechanism ensures equitable access to the shared resource.

The design satisfies the primary objectives of fairness, correctness, and reliable grant generation.

11. Applications

Round Robin Arbiters are commonly used in:

- Shared communication buses
 - Memory controllers
 - Network routers
 - FPGA interconnects
 - DMA controllers
 - Multi-core processor communication
 - Embedded systems
-

12. Advantages

- Prevents starvation.
 - Ensures fair allocation.
 - Simple hardware implementation.
 - Deterministic arbitration behavior.
 - Suitable for synchronous digital systems.
 - Easily scalable for additional requesters.
-

13. Limitations

- Every requester receives equal priority regardless of urgency.
 - High-priority real-time tasks cannot preempt lower-priority transactions.
 - Arbitration delay increases as the number of requesters increases.
-

14. Future Enhancements

Possible improvements to the design include:

- Support for 8 or 16 requesters.

- Parameterized Verilog implementation for configurable numbers of requesters.
 - Weighted Round Robin scheduling.
 - Dynamic priority adjustment.
 - Timeout-based preemption for long transactions.
 - Integration with standard bus protocols such as Wishbone, AXI, or AHB.
-

15. Conclusion

This project successfully implemented a synchronous 4-requester Round Robin Arbiter using Verilog HDL. A Finite State Machine was employed to manage the arbitration process, while a rotating priority mechanism ensured fair resource allocation among all requesters. Simulation in Xilinx Vivado verified the correct operation of the design under various request scenarios, including simultaneous requests and continuous bus occupancy.

The project demonstrates the practical application of digital design concepts such as FSM design, arbitration logic, register-based control, and hardware verification. The implemented arbiter provides a reliable and scalable solution for managing shared hardware resources in digital systems.

Tools and Technologies Used

- **Hardware Description Language:** Verilog HDL
- **Simulation Tool:** Xilinx Vivado Simulator
- **Design Methodology:** Register Transfer Level (RTL) Design
- **Verification:** Behavioral Simulation
- **Architecture:** Finite State Machine (FSM) based Round Robin Arbitration